

COST & GOVERNANCE

The Quiet Leak

A practical field guide to finding and fixing AI spend you never budgeted for.

IN THIS PAPER

01 Why AI spend behaves differently
02 The four leaks we look for first
03 What to record on every call
04 Start with a 30-day baseline
05 Cache and routing economics, briefly

October 2026 CloudyAI Research Free to download - no email required

SUMMARY

Why AI budgets fail slowly rather than loudly, the four leak classes we look for first, and the monthly cadence that keeps the number honest.

Most AI budgets do not fail in one dramatic overspend. They leak. A premium model answers a question a small one could have handled. The same 4,000-token policy document is pasted into a prompt forty times a day. A retry loop nobody logged quietly triples the cost of one failed integration. Each of these is small enough to survive a monthly review and large enough, over a year, to pay for the team that would have prevented it.

This paper is the working method we use to find and close those leaks. It assumes you already have an invoice, a product team, and no dedicated FinOps function. It does not assume you can change vendors, rewrite the product, or hire anyone.

Why AI spend behaves differently

Traditional software cost is mostly fixed. You buy seats, you reserve servers, and the bill is a function of headcount and time. AI inference is variable: cost is a function of how many tokens pass through a model, which model handles them, and how often the same context is sent again. A single product decision - adding a summarise button, widening a retrieval window, letting an agent loop three times instead of once - can move the bill by 30 per cent before anyone opens a spreadsheet.

That variability is not the real problem. The real problem is instrument mismatch: engineering sees requests, finance sees one consolidated line item, and nobody sees cost per feature. The invoice is the worst possible instrument for managing AI spend, because by the time it arrives the behaviour that produced it is already a month old.

> **The one-sentence rule**
>
> If you cannot attribute last month's spend to a feature and a model, you cannot optimise it. Instrumentation comes before cleverness.

The four leaks we look for first

1. Cacheable repetition

Repeated prompts are the most common and least glamorous leak. It shows up as a large static prefix - policy text, a product catalogue, a long system prompt, few-shot examples - sent unchanged on every call. Modern providers price cache reads at a fraction of standard input, often around a tenth, with a premium on the first write. If a prompt has 3,000 fixed tokens and 200 variable ones, you are paying full price for 94 per cent of your context every single time.

The fix is usually structural rather than technical: put everything stable at the start of the prompt and everything variable at the end, remove timestamps and random IDs from system prompts (they defeat prefix matching), and stop interpolating the user's name into the first line.

2. Tier mismatch

Top-tier models are routinely used for classification, extraction, reformatting, and short answers. These tasks are often solved at equal quality by a model one or two tiers down, at a tenth to a twentieth of the cost. The smell is specific: premium models returning fewer than 300 output tokens. That does not mean every short answer should be routed down - some short answers require real reasoning - but it means the question is worth asking with evidence rather than instinct.

Do not route on vibes. Take 100 real requests from production, run them through both tiers, and have a human grade the outputs blind. If the cheaper tier wins or ties on 90 of them, route the task, not the whole product.

3. Retry storms

An integration that fails and retries three times with the full context pays four times for one useful outcome. Add agent loops and tool calls that re-send the entire conversation on every step, and the effective cost per task can be an order of magnitude above the estimate. Log retries as first-class events with their own cost field. If retries exceed five per cent of calls, that is an engineering bug, not a pricing problem.

4. Unattributed and shadow usage

Personal API keys on corporate cards, prototypes that quietly became internal tools, a partner integration billed to your account, a notebook running a nightly batch against the production key. None of it is malicious, all of it is invisible, and it is often the largest single line in the first audit.

What to record on every call

You do not need a data warehouse to start. A structured log line per call, shipped to cheap storage, answers 90 per cent of the questions you will have. The fields below are the minimum we would accept before optimising anything.

Field	Why it earns its place
model + version	Prices and behaviour change between versions. Without the version you cannot explain a cost jump.
tokens_in / tokens_out	The actual unit of spend. Character counts and word counts lie.
timestamp	Turns a monthly total into a shape you can question: spikes, gaps, weekend batch jobs.
feature or endpoint	The only way to answer 'what is costing us money' with a name instead of a shrug.
cost	Store it at write time. Recomputing from a price sheet months later is how audits go wrong.
cache hit / miss	Tells you whether the caching you paid for is actually happening.
outcome or error	Retries and failures are invisible in aggregate token counts.
prompt version	A prompt change is a cost change and a quality change. Treat it like a deploy.

Start with a 30-day baseline

01 Pick one high-volume feature. Not the most expensive - the one with the clearest logs.

02 Log every call for 30 days with the fields above. Resist the urge to optimise during this window.

03 Compute three numbers: cost per successful task, tokens per successful task, and share of spend from the top ten prompt shapes.

04 Identify the dominant leak class. In our experience it is repetition about half the time, tier mismatch about a third, and everything else the remainder.

05 Change one thing. Re-measure for two weeks. Keep or revert on evidence.

Three days of data is not a baseline. Weekly traffic patterns, month-end batches, and Monday morning spikes are where the surprises live, and a short sample will hide exactly the behaviour you are trying to find.

Cache and routing economics, briefly

Caching pays when a prefix is large, stable, and reused within the cache window. It does not pay when your prefixes are short, your traffic is sparse, or every user carries a different context. A rough test: multiply the static prefix tokens by the number of daily calls. If that number is in the millions, caching is worth engineering time this week. If it is in the thousands, fix something else first.

Routing pays when the quality gap between tiers is smaller than the price gap on your actual task. The two failure modes are equally expensive: routing everything down and quietly degrading the product, or routing nothing down and paying a premium for reformatting JSON. Build a small fixed evaluation set of real requests with human grades, and re-run it every time you change a route.

The monthly cadence

Rhythm	What happens	Owner
Weekly, 15 minutes	Spend against the previous week; alert review; anything above the 95th percentile of daily spend gets a name attached.	Engineering lead
Monthly, 1 hour	Routing audit against the eval set; cache hit rate by feature; retry rate; top ten prompt shapes by cost.	Product + engineering
Quarterly, half day	Repricing review against current provider rates; renegotiate committed spend; retire dead features that still receive traffic.	Finance + leadership

The cadence matters more than the sophistication. A one-page review every month will beat a beautiful dashboard nobody opens.

Signals worth alerting on

Signal	Healthy range
Cost per successful task, week on week	flat or falling
Cache hit rate on cacheable prefixes	above 80%, or dropping overnight
Retries as a share of calls	under 2%
Premium-tier share of spend	matched to a task mix
Unattributed spend	zero

Signal	Investigate
Cost per successful task, week on week	rising 20% with no launch
Cache hit rate on cacheable prefixes	below 50%, or dropping overnight
Retries as a share of calls	above 5%
Premium-tier share of spend	premium tier above 70% of calls
Unattributed spend	any key with a nonzero balance

What this does not solve

Cost governance cannot fix an unclear product. If a feature has no defined success metric, a cheaper model will simply produce the same ambiguous outcome for less money. It also cannot substitute for vendor negotiation at scale, and it will not survive a reorganisation unless the log schema is owned by someone who stays.

The good news is that the first pass is cheap. A week of instrumentation and an afternoon of honest reading will usually find more than a quarter of the bill. The rest is discipline.

USE THE TOOL, NOT THE PAPER

Our AI spend audit runs entirely in your browser: drop in a usage export, get a directional read on cacheable repetition and tier mismatch. The file never leaves your machine. It is a starting point for the conversation above, not a replacement for thirty days of good logs.

ABOUT THIS PAPER

CloudyAI is a Zimbabwean AI studio. We deploy computer vision on existing plant cameras, audit AI spend, and help teams respond when an AI system misbehaves. Research is published free, without an email wall.

Corrections and questions: research@cloudyai.co.zw. If you would like this method applied to your plant, the first conversation is free and the camera audit is a fixed-price week.